

VC2 – Algoritmos de Ordenación

03MAIR – Algoritmos de Optimización

Agenda

1. Algoritmos de ordenación

- Inserción
- Burbuja
- Selección
- Merge Sort
- Heap Sort
- Quick Sort
- Radix Sort

Foro

03MIAR_10_A_2025-26_Algoritmos de Optimización

Contenido Calendario Anuncios **Debates** Libro de calificaciones Grupos Logros

Foros de dudas y consultas

Dudas sobre el contenido de la asignatura

Fecha de entrega: 6/2/26 14:00 (CET)

Este foro sirve para resolver todas aquellas dudas o cuestiones relacionadas con la asignatura

Dudas sobre las actividades y la evaluación final

Fecha de entrega: 19/12/25 14:02 (CET)

Foros para estudiantes

Presentación de los estudiantes

Espacio para presentarse al resto de compañeros/as

Foro. Elementos comunes en dos listas



Carlos Javier Bravo Intriaño

18 de dic. de 2025 5:40



Hola a todos,

Siguiendo el debate interesante que tuvimos en la primera clase sobre el ejercicio de listas, el profesor me encomendó profundizar un poco más en por qué ciertas soluciones presentan un comportamiento cuadrático y otras lineal.

Preparé este notebook donde comparo el rendimiento de la solución con ``list comprehension``, la variante del bucle ``for`` y la de ``set``. Los resultados confirman el impacto de los bucles anidados en listas grandes, pero también destacan que las soluciones basadas en iterar listas eran más amigables con la memoria RAM.

Les dejo aquí los resultados y también incluí una solución extra que intenta buscar un equilibrio. Me encantaría conocer sus opiniones, correcciones, feedback o si ven algún detalle que se me haya pasado por alto.

<https://colab.research.google.com/drive/1TSyWUitOgS7Cf8TBK5rnsyKRYjkyYslx>

Como dato extra, ya que en la clase de hoy tocamos algo sobre el algoritmo de Heron, la ecuación iterativa para encontrar la raíz cuadrada se puede conseguir también con el método de newton-rapson, en el siguiente enlace propongo una versión donde encuentro un valor inicial o un ``magic number`` para que en una o dos iteraciones se pueda encontrar un resultado muy aproximado con un error absoluto cercano a 10^{-5} .

<https://leetcode.com/problems/sqrtx/solutions/6071226/fast-sqrt-in-python/>

<https://aliennerd.dev/sqrt>

¡Felices Fiestas!

Foro. Elementos comunes en dos listas

- ¿Realmente Carlos consigue $O(n)$ con hash (Complejidad manual) ?
- ¿En todos los casos?
- ¿Y que es un hash?
- Y si no tenemos en nuestro lenguaje algo como hash ?
- Y si ordenamos previamente las 2 listas ?



Carlos Javier Bravo Intriago
18 de dic. de 2025 5:40

Hola a todos,

Siguiendo el debate interesante que tuvimos en la primera clase sobre el ejercicio de listas, el profesor me encomendó profundizar un poco más en por qué ciertas soluciones presentan un comportamiento cuadrático y otras lineal.

Preparé este notebook donde comparo el rendimiento de la solución con `'list comprehension'`, la variante del bucle `'for'` y la de `'set'`. Los resultados confirman el impacto de los bucles anidados en listas grandes, pero también destacan que las soluciones basadas en iterar listas eran más amigables con la memoria RAM.

Les dejo aquí los resultados y también incluí una solución extra que intenta buscar un equilibrio. Me encantaría conocer sus opiniones, correcciones, feedback o si ven algún detalle que se me haya pasado por alto.

<https://colab.research.google.com/drive/1TSyWUitOg57Cf8TBK5rnsyKRVjkyxslx>

Como dato extra, ya que en la clase de hoy tocamos algo sobre el algoritmo de Heron, la ecuación iterativa para encontrar la raíz cuadrada se puede conseguir también con el método de newton-rapson, en el siguiente enlace propongo una versión donde encuentro un valor inicial o un `'magic number'` para que en una o dos iteraciones se pueda encontrar un resultado muy aproximado con un error absoluto cercano a 10^{-5} .

<https://leetcode.com/problems/sqrtx/solutions/6071226/fast-sqrt-in-python/>

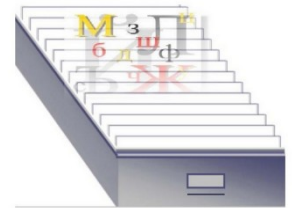
<https://allennerd.dev/sqrt>

¡Felices Fiestas!

Algoritmos de Ordenación

Algoritmos de Ordenación

- **Definición:** Ordenar una lista de valores numéricos
- Algoritmos:
 - ✓ Por Inserción
 - ✓ Por Selección
 - ✓ Burbuja
 - ✓ Mezcla
 - ✓ Montículos
 - ✓ Ordenación rápida (Quick Sort)
- Utilizaremos algunas **operaciones no elementales**:
 - ✓ Obtener el máximo o el mínimo de una lista : $O(n)$
 - ✓ Intercambiar dos valores de una lista: $O(1)$



Algoritmos de Ordenación(I)

✓ Inserción

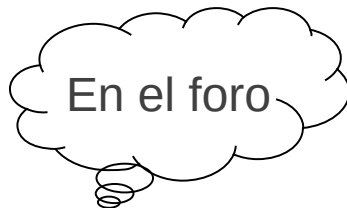
- ✓ Se recorre la lista completa para en cada iteración construir una sub-lista ordenada.
- ✓ Aconsejable para listas pequeñas y parcialmente ordenadas.
- ✓ Es el método que usamos manualmente.
- ✓ Complejidad (varia con la disposición inicial):
 - $O(n)$ para el caso más favorable
 - $O(n^2)$ para el caso más desfavorable
- ✓ Aconsejable para **listas pequeñas y parcialmente ordenadas**

8	7	5	9	1	6	2	4	3
7	8	5	9	1	6	2	4	3
5	7	8	9	1	6	2	4	3
5	7	8	9	1	6	2	4	3
1	5	7	8	9	6	2	4	3
1	5	6	7	8	9	2	4	3
1	2	5	6	7	8	9	4	3
1	2	4	5	6	7	8	9	3
1	2	3	4	5	6	7	8	9

Algoritmos de Ordenación(II)

✓ Burbuja

- ✓ Se recorre la lista para intercambiar cada elemento con el adyacente(si es preciso) en cada iteración, reduciendo progresivamente(de uno en uno) el tamaño de la lista.
- ✓ Se puede mejorar deteniendo el proceso si no hay intercambios
- ✓ Complejidad: siempre $O(n^2)$ independientemente de la disposición inicial(excepto por mejoras)

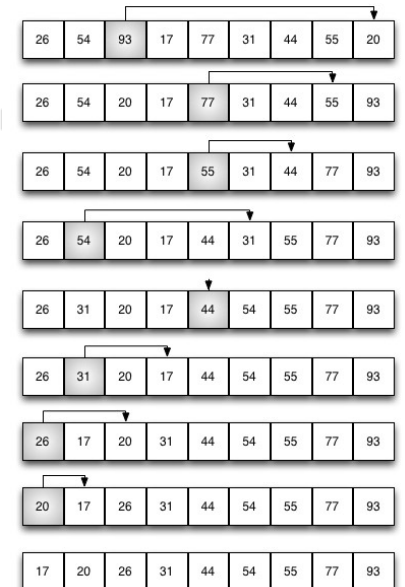


54	26	93	17	77	31	44	55	20
26	54	93	17	77	31	44	55	20
26	54	93	17	77	31	44	55	20
26	54	17	93	77	31	44	55	20
26	54	17	77	93	31	44	55	20
26	54	17	77	31	93	44	55	20
26	54	17	77	31	44	93	55	20
26	54	17	77	31	44	55	93	20
26	54	17	77	31	44	55	20	93

Algoritmos de Ordenación(III)

✓ Selección

- ✓ Se recorre la lista para encontrar el máximo (o el mínimo) en cada iteración y situarlo en la posición que le corresponde, reduciendo progresivamente(de uno en uno) el tamaño de la lista.
- ✓ También es un método usado manualmente.
- ✓ Complejidad: siempre $O(n^2)$ independientemente de la disposición



Algoritmos de Ordenación(IV)

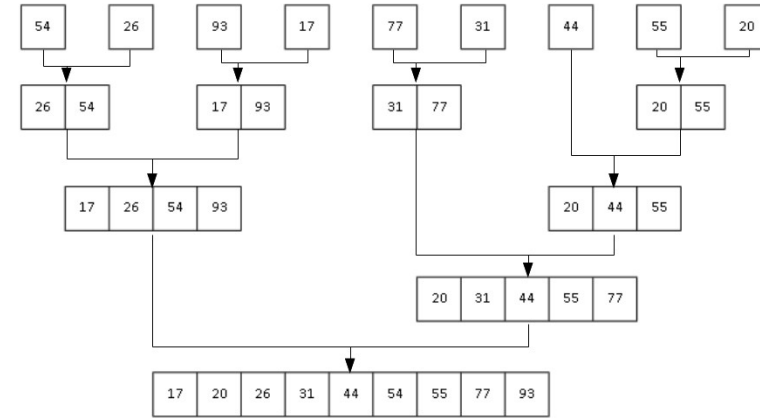
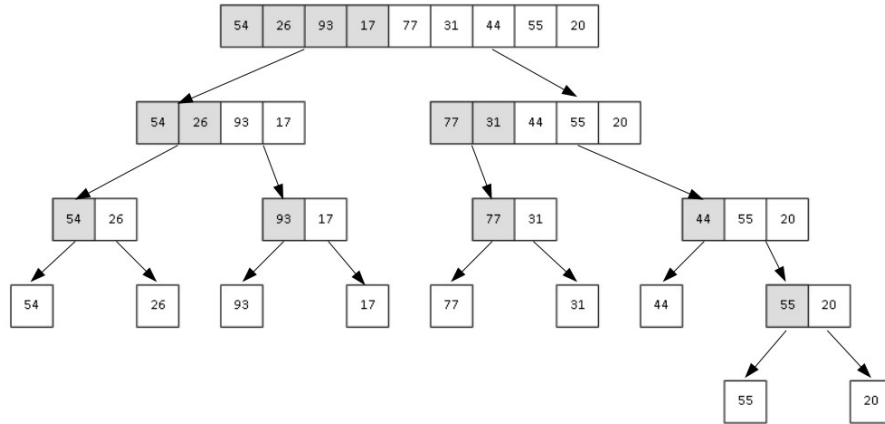
✓ Mezcla(Merge Sort)

- ✓ Divide recursivamente la lista en dos mitades y combina los resultados
- ✓ Basado en la técnica “**Divide y Vencerás**”
- ✓ Muy usado en el pasado dado que se adapta bien al acceso secuencial.
- ✓ Complejidad: siempre $O(n \cdot \log(n))$ en todos los casos
- ✓ El cálculo se realiza por **ecuaciones en recurrencia**



Algoritmos de Ordenación(V)

✓ Mezcla(Merge Sort)



Algoritmos de Ordenación(VI)

✓ Montículos(Heap Sort)

- ✓ Construye un árbol binario completo donde el valor del padre es menor que el de sus hijos.
- ✓ En cada iteración se coloca un nuevo nodo reajustando recursivamente el resto si es necesario.
- ✓ Complejidad: siempre $O(n \cdot \log(n))$ en el peor caso aunque mejora en casos favorables.
- ✓ Se comporta bien en casos desfavorables y muy bien en favorables y sin coste de almacenamiento extra.
- ✓ Algo complicado de implementar.

6 5 3 1 8 7 2 4

<https://d18l82el6cdm1i.cloudfront.net/uploads/uv9rgMfetq-heapsort-example.gif>

Algoritmos de Ordenación(VII)

✓ Ordenación Rápida(Quick Sort)

✓ Proceso en dos partes:

- 1) Divide recursivamente la lista inicial en dos de tal manera que todos los elementos de la primera lista son menores que los de la segunda
- 2) Se combinan las dos listas

✓ Basado en la técnica “Divide y Vencerás”

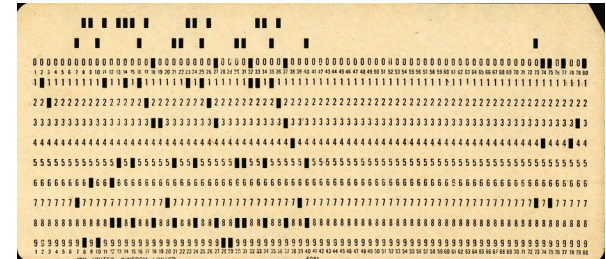
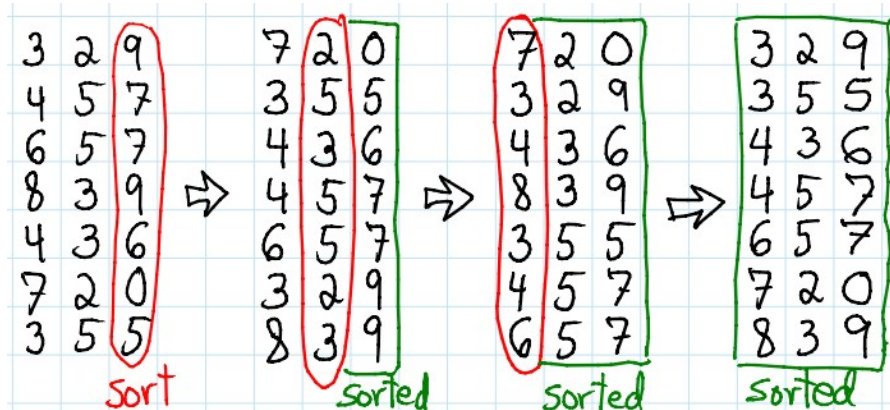
✓ Complejidad: $O(n^2)$ en el peor caso y $O(n \cdot \log(n))$ en el mejor caso o en promedio.

✓ Los casos favorables no dependen de los datos sino de la elección del pivote para dividir las listas.

Algoritmos de Ordenación(VIII)

✓ Radix Sort

- ✓ Ordenaciones parciales por dígitos de más a menos significativos(o al contrario).
- ✓ Creado en 1887. Se usó en los inicios de la computación para clasificar tarjetas perforadoras.
- ✓ Complejidad $O(n \cdot w)$ | $n=n^\circ$ de elementos y w =el tamaño de los elementos.
- ✓ Adecuada para elementos alfanuméricos.



Algoritmos de Ordenación(IX)



- Resumen de complejidades:

Algoritmo	Mejor	Peor	Medio	Espacio
Ordenar mezcla	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(n)$
Ordenación por inserción	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Ordenación por burbuja	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Ordenación rápida	$O(n \log(n))$	$O(n^2)$	$O(n \log(n))$	$O(1)^a$
Ordenación por montículos	$O(n \log(n))$	$O(n \log(n))$	$O(n \log(n))$	$O(1)$

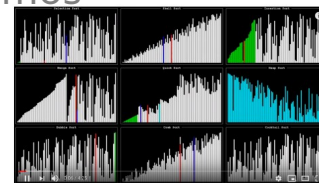
^a En el mejor caso incluso es $\log(n)$.

Análisis comparativo de algoritmos de ordenamiento

<https://pereiratechtalks.com/analisis-de-algoritmos-de-ordenamiento/>

- Otras ordenaciones

- ✓ Shell: mejora a inserción por comparación con varias posiciones.
- ✓ Burbuja bidireccional (cocktail sort). Ordena por los dos extremos
- ✓ Timsort, Tree Sort, Bucket Sort, Radix Sort, ...



<https://www.youtube.com/watch?v=ZZuD6iUe3Pc>

Algoritmos de Ordenación(X)

Comparación de ordenaciones Quick Sort y Heap Sort(montículo)



International Journal of Computing Academic Research (IJCAR)
ISSN 2305-9184 Volume 3, Number 2(April 2014), pp. 39-57
© MEACSE Publications
<http://www.meacse.org/ijcar>

Comparative Performance Evaluation of Heap-Sort and Quick-Sort Algorithms

Ebtisam S. Al-Jaloud, Haifaa A. Al-Aqeel and Ghada H. Badr
King Saud University. Riyadh, Saudi Arabia

Abstract

Sorting is a fundamental operation in computer science (many programs use it as an intermediate step). A large number of sorting algorithms have been made in order to have a best performance in terms of computational complexity (best, average and worst), memory usage, stability and method. One of sorting algorithms can perform much better than another. For that, we intend to make a comparison research paper and two representative algorithms were chosen to show these differences. In the paper, a comparative performance evaluation of two sorting algorithms Heap-Sort and Quick-Sort will be presented depending on the performance factors shown above. Furthermore, the most significant enhancements of these algorithms will be discussed.

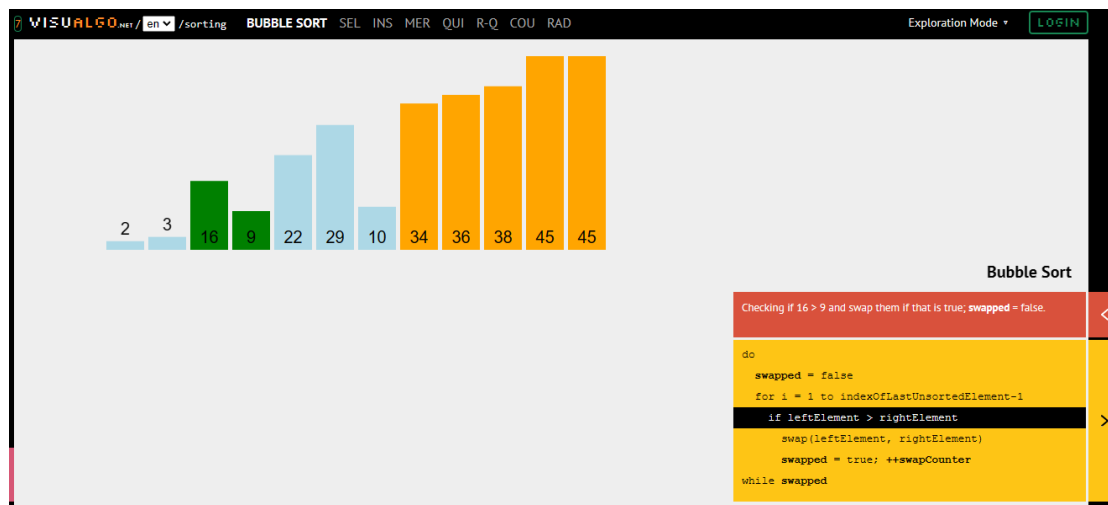
Keywords: HeapSort, QuickSort, pivot

<http://www.meacse.org/IJCAR/archives/41.pdf>

<https://web.archive.org/web/20130801175915/http://www.cs.auckland.ac.nz/~jmor159/PLDS210/qsor3.html>

Algoritmos de Ordenación(X)

Implementación



<https://visualgo.net/en/sorting?slide=1>

Algoritmos de Ordenación(XI)

- **Resumen de aplicaciones.¿Donde se utiliza?:**
 - ✓ Ordenar resultados
 - ✓ Realizar búsquedas
 - ✓ Encontrar la mediana o el par mas cercano
 - ✓ Identificar “outliers” / anomalías
 - ✓ Detectar duplicados
 - ✓ ...
 - ✓ Compresión de datos
 - ✓ Planificación de tareas
 - ✓ Simulaciones
 - ✓

Diseño de Algoritmos . Implementación(javascript) y Visualización

The screenshot displays the Algorithm Visualizer interface for the N-Queens Problem. The left sidebar lists various algorithms, with 'N-Queens Problem' selected. The main area shows a 4x4 grid representing the chessboard, with the current state of the board visualized. The grid shows the positions of queens for the first three rows: Row 0 has a queen at column 1, Row 1 at column 3, and Row 2 at column 2. The fourth row is currently empty. The 'Queen Positions' section below the grid shows the current state: Row 0: 1, Row 1: 3, Row 2: 2, Row 3: 2. The 'Progress' section shows the current iteration: 'new iteration of nQueens () with currentQueen = 4 & currentCol = 4'. The right panel displays the JavaScript code for the implementation, which includes functions for generating the board, validating the state, and visualizing the solution.

```
1: // Import visualization libraries
2:
3: const N = 4; // Just change the value of N and the visuals will reflect the configuration!
4: const board = function createBoard(N) {
5:   const result = [];
6:   for (let i = 0; i < N; i++) {
7:     result[i] = Array(...Array(N)).map(Number.prototype.valueOf, 0);
8:   }
9:   return result;
10: }
11:
12: const queens = function setup(N) {
13:   const result = [];
14:   for (let i = 0; i < N; i++) {
15:     result[i] = [-1, -1];
16:   }
17:   return result;
18: }
19:
20: // Define tracer variables
21:
22: function validateState(row, col, currentQueen) {
23:   for (let q = 0; q < currentQueens; q++) {
24:     const currentQ = queens[q];
25:     if (row === currentQ[0] || col === currentQ[1] || (Math.abs(currentQ[0] - row) === Math.abs(currentQ[1] - col))) {
26:       return false;
27:     }
28:   }
29:   return true;
30: }
31:
32: function nQ(currentQueen, currentCol) {
33:   // logger
34:   if (currentQueen >= N) {
35:     // logger
36:     return true;
37:   }
38:
39:   let found = false;
40:   let row = 0;
41:   while ((row < N) && !found) {
42:     // visualize
43:     if (validateState(row, currentCol, currentQueen)) {
44:       queens[currentQueen][0] = row;
45:       queens[currentQueen][1] = currentCol;
46:       // visualize
47:       found = nQ(currentQueen + 1, currentCol + 1);
48:     }
49:     if (!found) {
50:       // visualize
51:     }
52:     row++;
53:   }
54:   return found;
55: }
56:
57: // logger
58: nQ(0, 0);
59: // logger
```

Manual de la asignatura

03MIAR
ALGORITMOS
DE
OPTIMIZACIÓN

INICIO

Bienvenida

Calendario

INFORMACIÓN

01. Materiales docentes

**Manual de la asignatura**
Archivos adjuntos:  03MIAR_RReyero_nueva_imagen.pdf (5,02 MB)

**03MIAR | ALGORITMOS DE OPTIMIZACIÓN**
Explicación y práctica de las técnicas y métodos para diseñar y analizar algoritmos orientados a resolver problemas de optimización en el ámbito de...



Foro

03MIAR_10_A_2025-26_Algoritmos de Optimización

[Contenido](#) [Calendario](#) [Anuncios ¹](#) [Debates ¹](#) [Libro de calificaciones](#) [Grupos](#) [Logros](#)

 Foros de dudas y consultas 

 Dudas sobre el contenido de la asignatura 

Fecha de entrega: 6/2/26 14:00 (CET)

Este foro sirve para resolver todas aquellas dudas o cuestiones relacionadas con la asignatura

 Dudas sobre las actividades y la evaluación final

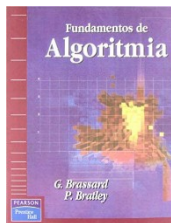
Fecha de entrega: 19/12/25 14:02 (CET)

 Foros para estudiantes 

 Presentación de los estudiantes

Espacio para presentarse al resto de compañeros/as

Bibliografía



Fundamentos de algoritmia: Una perspectiva de la ciencia de los computadores

Paul Bratley , Gilles Brassard

ISBN 13: 9788489660007



Introducción al diseño y análisis de algoritmos

R.C.T. Lee,...

ISBN 13: 9789701061244



Técnicas de diseño de algoritmos

Guerequeta, R., y Vallecillo, A.

<http://www.lcc.uma.es/~av/Libro/indice.html>

Bibliografía(en la biblioteca de la VIU)



Una introducción a las matemáticas para el análisis y diseño de algoritmos

Pérez Aguila, R. (2012).

<https://tinyurl.com/yzlt5oed>

¿Preguntas?



Gracias

raul.reyero@professor.universidadviu.com